EDGE AI / COMPUTER VISION / ROBOTICS

Design and Implementation of a Jetson-Based Real-Time Object Detection System with YOLO26 and ROS 2

English technical report sample for the LEARN-yolo project.

Working report sample

Project	LEARN-yolo
Active model	YOLO26m Tabletop v1
Target	NVIDIA Jetson Orin NX, 16 GB
Stack	Ultralytics, PyTorch, OpenCV, ROS 2 Foxy
Author	[Name / Student ID]
Institution	[Institution / Course]

Executive Summary

This project implements a complete object detection workflow from dataset preparation and YOLO training to cross-platform inference and deployment on an NVIDIA Jetson edge computer. The current model detects 11 tabletop object classes, publishes structured ROS 2 outputs, displays an OpenCV preview, and supports image and video capture.

0.845	0.647	0.841	8.20
TEST MAP50	TEST MAP50-95	TEST PRECISION	MEAN JETSON FPS

Current result. YOLO26m runs on the Jetson Orin NX with a USB camera and exposes detections, annotated images, and FPS as ROS 2 topics.

Draft boundary. A compact ROS 2 subscriber capture remains to be inserted as formal evidence. This sample marks that item instead of inventing an output.

Contents

1. Scope and Requirements

The early prototype detected keyboard, monitor, and mouse. The active project replaces that contract with an 11-class tabletop task. Legacy three-class metrics are not compared directly because the datasets and class definitions differ.

Functional requirements

- USB camera input and real-time YOLO inference.
- Native OpenCV annotated preview.
- Annotated snapshots and MP4 recordings.
- ROS 2 detections, images, FPS, and control services.
- One-command and desktop-shortcut startup.

Deployment requirements

- Preserve JetPack-compatible CUDA and PyTorch.
- Support PT, ONNX, and target-built TensorRT engines.
- Keep deployment settings outside Python source.
- Maintain Git-tracked models, data, code, and evidence.
- Allow preview-free headless operation.

Completion status

Capability	Status	Evidence
Dataset audit	**Complete**	Class map, split counts, audit summary
Training and independent test	**Complete**	Weights, metrics, curves, matrices
Jetson USB camera inference	**Complete**	Annotated video and runtime log
ROS 2 publication	**Implemented**	Publisher code and interactive validation
Report-ready subscriber capture	**Pending**	Short text artifact to be added

2. Dataset and Training

The `tabletop_v1` dataset is derived from Roboflow Universe project `table-03wsy`, version 1, under CC BY 4.0. Images are exported at 640 x 640 with no generated augmentation.

1,142	148	151	11
TRAIN IMAGES	VALIDATION IMAGES	TEST IMAGES	CLASSES

Split	Images	Boxes	Purpose
Train	1,142	1,926	Parameter optimization
Validation	148	234	Checkpoint selection
Test	151	259	Independent evaluation

Classes

book, bottle, earphone, glass, headphone, keyboard, laptop, mobile, mouse, pen, and penstand.

> **Audit result.** No missing pairs, unreadable images, invalid normalized boxes, empty classes, or exact cross-split duplicates were found. The upstream spelling `bottole` was corrected to `bottle` without changing numeric labels.

Training configuration

Setting	Value	Setting	Value
Base checkpoint	yolo26m.pt	Input size	640
Batch size	16	Seed	42
Requested epochs	100	Recorded rows	Epochs 1-99
Best CSV row	Epoch 79	Ultralytics	8.4.135

> **Final-report check.** The requested epoch count and the 99 recorded CSV rows must be reconciled before submission.

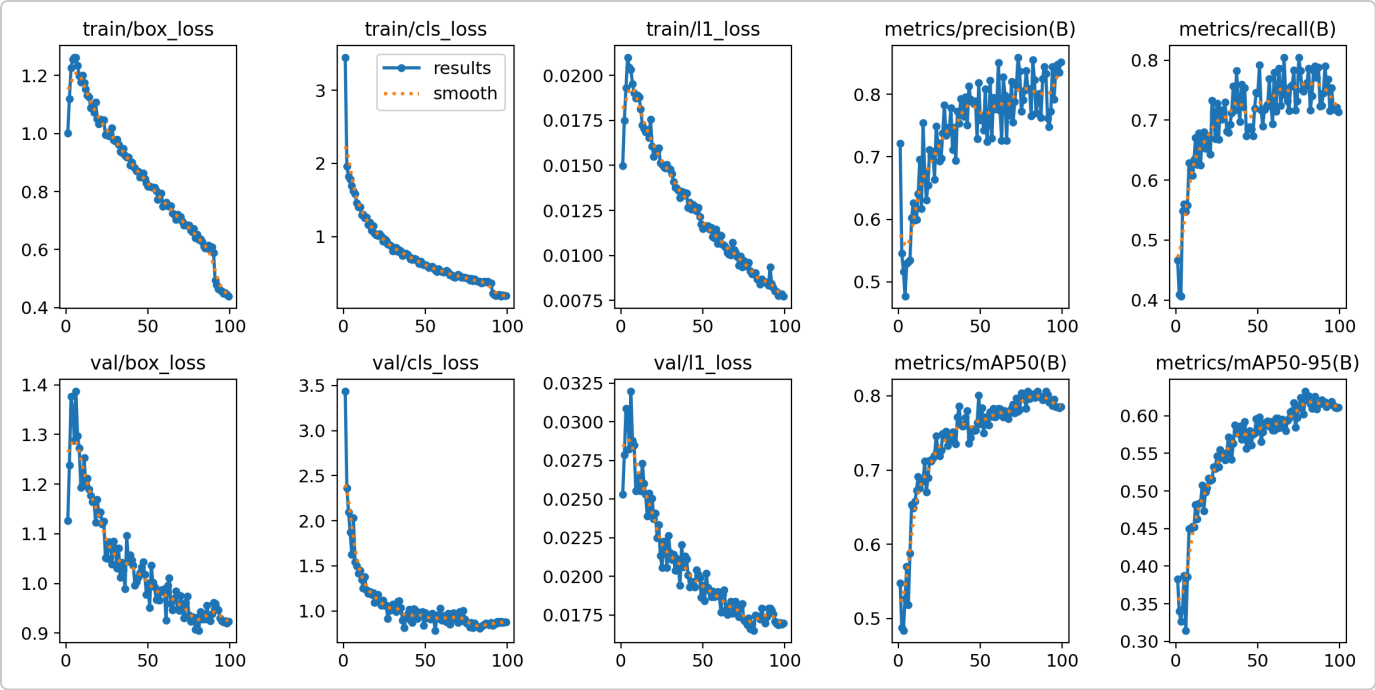

Figure 1. YOLO26m training losses and validation metrics.

3. System Architecture

Frame acquisition, inference, ROS message conversion, visualization, and media capture are separated into focused components.

1. Read a frame and create a ROS timestamp and frame ID.
2. Run Ultralytics prediction with configured thresholds.
3. Convert class IDs, confidence values, and boxes into `vision_msgs`.
4. Publish detections and FPS; optionally publish and display annotated images.
5. Handle snapshot and recording services using annotated frames.

Model artifacts

Format	Use	Project role
PT	PyTorch training and inference	Current Jetson runtime
ONNX	Portable CPU/GPU inference	Windows, Linux, and macOS application
TensorRT engine	NVIDIA-optimized inference	Future Jetson performance option; build on target

Core modules

Module	Responsibility
`detector_node.py`	ROS orchestration, YOLO inference, publishers, and services
`camera_stream.py`	Camera acquisition and lifecycle
`ros_messages.py`	Image conversion and ROS message compatibility
`opencv_view.py`	Native OpenCV preview
`media_capture.py`	JPEG snapshots and MP4 recording
`start_detector.sh`	Checks, build, configuration, and startup

4. Independent Test Results

0.841	0.813	0.845	0.647
PRECISION	RECALL	MAP50	MAP50-95

Class	P	R	mAP50	mAP50-95
book	.593	.657	.663	.509
bottle	.949	.949	.973	.819
earphone	.781	.715	.825	.425
glass	.903	.818	.810	.656
headphone	.902	.857	.893	.720
keyboard	.949	.925	.943	.782
laptop	.924	.875	.912	.761
mobile	.708	.714	.788	.563
mouse	.959	.870	.902	.799
pen	.642	.615	.606	.295
penstand	.945	.946	.981	.784

Bottle, keyboard, laptop, mouse, and penstand are strong classes. Pen is the clearest weakness under stricter localization criteria and should receive additional hard examples.

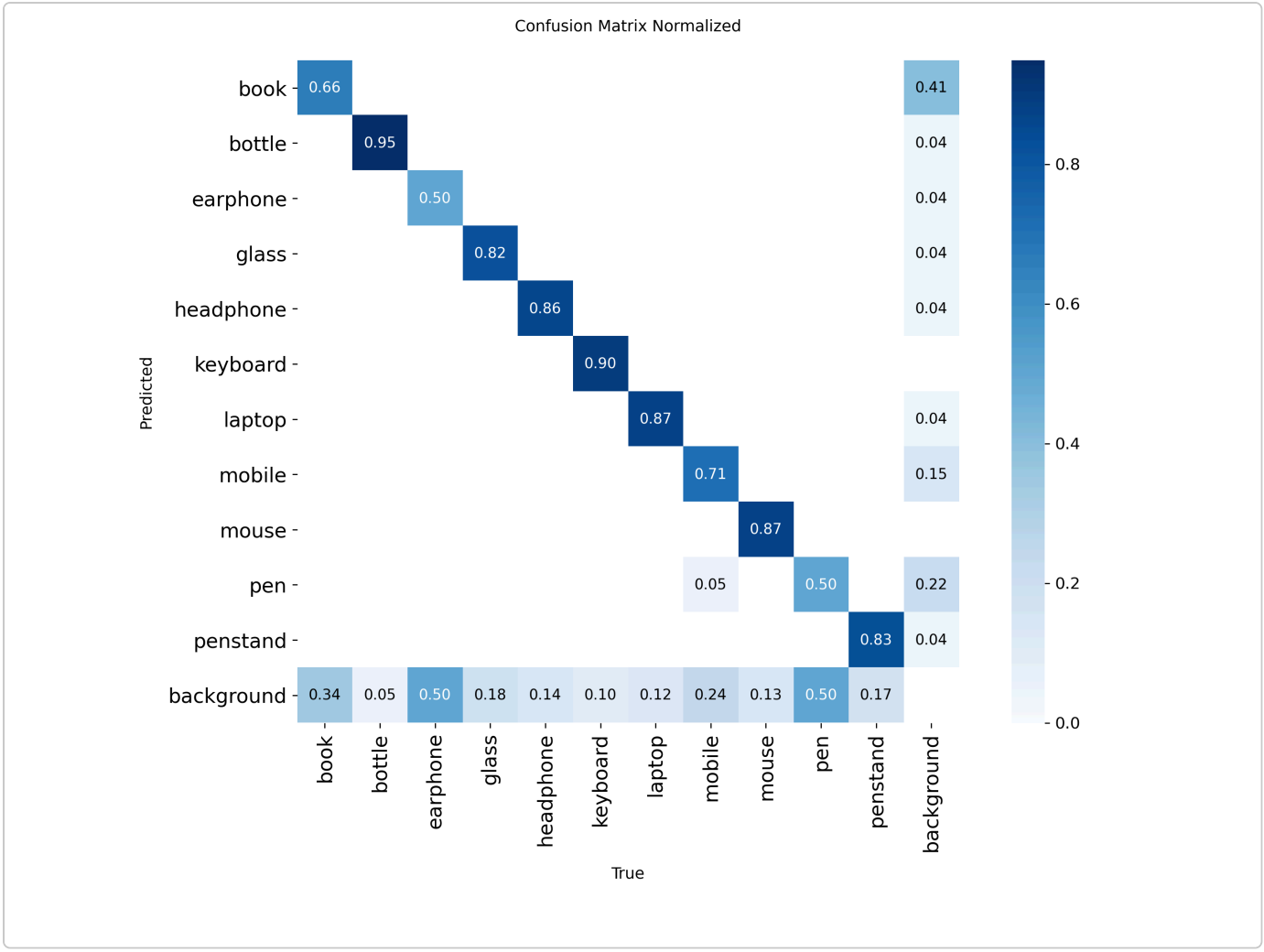

Figure 2. Normalized confusion matrix for the independent test split.

5. ROS 2 Interface

Recognition results are published through DDS rather than printed to the detector terminal.

Topic	Type	Content
`/yolo/detections`	`vision_msgs/Detection2DArray`	Class, confidence, timestamp, frame ID, pixel box
`/yolo/annotated_image`	`sensor_msgs/Image`	BGR8 annotated image
`/yolo/fps`	`std_msgs/Float32`	Smoothed output FPS

Service	Purpose
`/yolo/save_snapshot`	Save the latest annotated frame
`/yolo/set_recording`	Start or stop annotated recording

```
boxes.xyxy -> bbox center and size
boxes.conf -> results[0].score
boxes.cls  -> model class metadata -> results[0].id
ROS clock  -> header.stamp
```

> **Pending evidence.** Before final submission, insert one concise captured `/yolo/detections` message and a topic-rate measurement. Do not fill this section with repeated YAML.

```
Illustrative schema only:
header: {frame_id: camera_optical_frame}
detections:
  - results: [{id: mouse, score: 0.87}]
    bbox: {center: {x: ..., y: ...}, size_x: ..., size_y: ...}
```

6. Jetson Deployment

Hardware	Configuration
Device	Jetson Orin NX Developer Kit
Memory	16 GiB
Architecture	ARM64 / aarch64
Camera	USB UVC

Software	Version
Ubuntu	20.04.6 LTS
L4T	R35.6.5
CUDA	11.4
TensorRT	8.5.2.2
ROS 2	Foxy

Observed live performance

8.20	6.90	9.09	31
MEAN FPS	MINIMUM FPS	MAXIMUM FPS	LOG SAMPLES

The PyTorch model provides functional edge inference for demonstration and robotics prototyping. A Jetson-built FP16 TensorRT engine is the main route to higher throughput.

> **Reporting convention.** Performance is taken from detector runtime logs. Recording timestamps and video duration are excluded.

Startup

```
cd ~/projects/LEARN-yolo
git pull --ff-only
./deploy/jetson/start_detector.sh --view
```

7. Live Demonstration

Representative annotated frames show mobile, pen, and mouse detections under position changes, hand occlusion, and multi-object scenes.

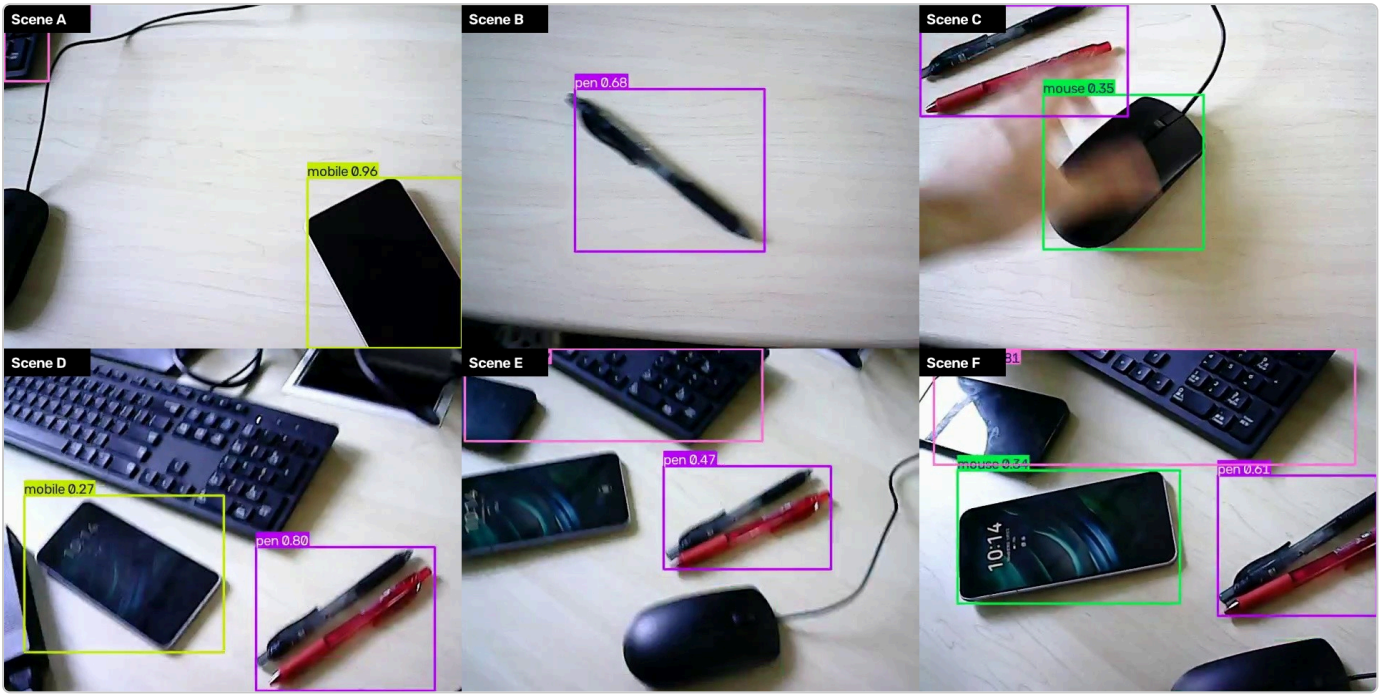

Figure 3. Representative frames from the Jetson USB camera demonstration.

Strengths

- Boxes follow objects as positions change.
- Multiple classes can be shown simultaneously.
- Mouse behavior matches its strong test result.

Weaknesses

- Confidence drops near edges and occlusions.
- Small pen localization is less stable.
- The video demonstrates representative, not all 11, classes.

The qualitative observations agree with the independent test results, especially the low pen mAP50-95 and strong mouse performance.

8. Limitations and Conclusion

Limitations and next steps

1. Collect more cluttered, occluded, and motion-blurred pen and mobile examples.
2. Broaden live acceptance coverage beyond the representative classes in the current video.
3. Benchmark a target-built FP16 TensorRT engine against the same scene.
4. Archive a compact ROS 2 subscriber capture and topic-rate result.
5. Reconcile the intended epoch count with the preserved CSV rows.

Conclusion

The project has advanced from model training to a functional edge robotics application. YOLO26m Tabletop v1 achieves 0.845 mAP50 and 0.647 mAP50-95 on an independent 11-class test set and runs on the Jetson Orin NX at an observed mean of approximately 8.20 FPS.

The implementation provides OpenCV visualization, media capture, one-command startup, and ROS 2 publishers for detections, annotated images, and performance data. It is ready for a full report draft; final submission should add concise ROS 2 subscriber evidence and reconcile the training epoch description.

Core artifact index

Artifact	Repository path
Dataset	`datasets/tabletop_v1/`
Model and metrics	`weights/yolo26m_tabletop_v1/`
ROS 2 package	`ros2_ws/src/yolo_detector/`
Jetson launcher	`deploy/jetson/start_detector.sh`

Working sample. Replace author placeholders and insert final ROS 2 evidence before submission.